Institut d'Optique Graduate School

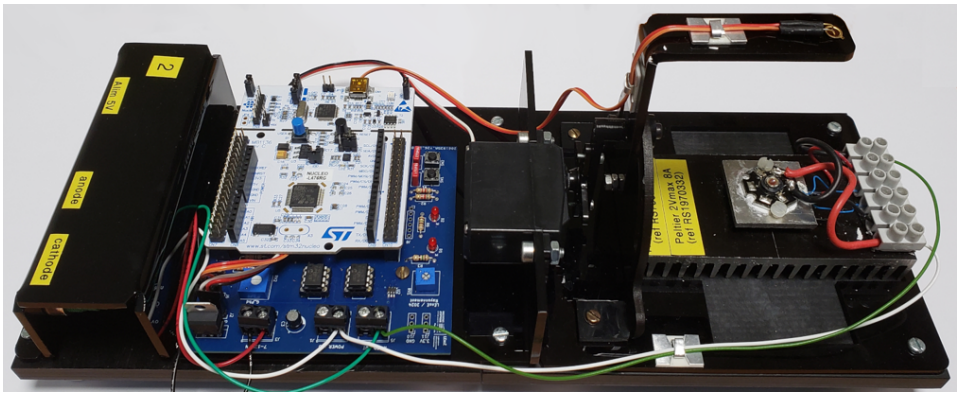
INTERFAÇAGE NUMÉRIQUE

Travaux Pratiques

Semestre 6

Banc de mesure de rayonnement

2 séances



Ce sujet est disponible au format électronique sur le site du LEnsE - https ://lense.institutoptique.fr/ dans la rubrique Année / Première Année / Interfaçage Numérique S6 / Bloc 3 – Interface d'acquisition de données.

© 2025 by LEnsE-IOGS

Banc de mesure de rayonnement

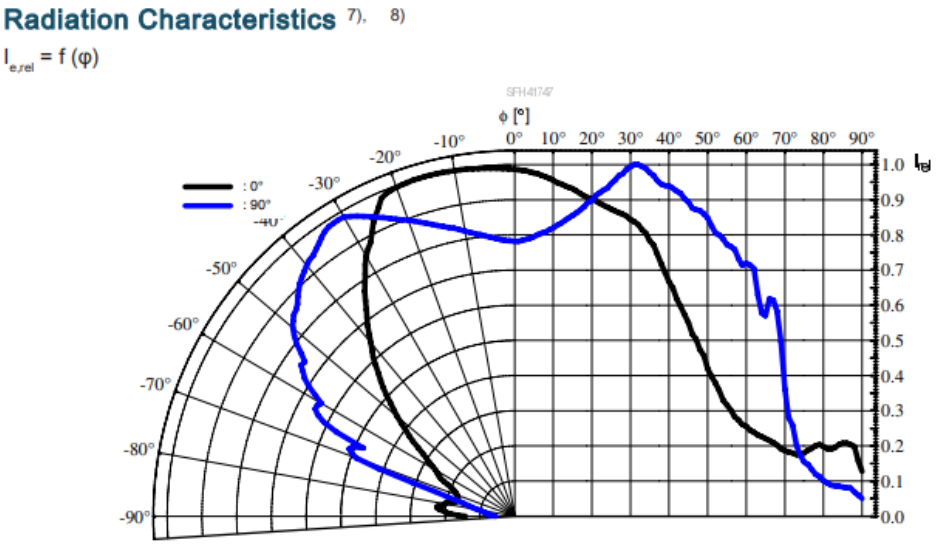
À l'issue des séances de TP concernant le **bloc d'acquisition d'un diagramme de rayonnement**, les étudiant·es seront capables de :

- Développer, mettre en œuvre et valider une **solution informatique pour acquérir des données** selon un protocole établi
- Mettre en œuvre un **protocole simple de communication** entre un ordinateur et un microcontrôleur pour transmettre des commandes et lire des données
- Documenter un programme informatique
- Etablir un **protocole de test**, réaliser les tests et rédiger une **analyse préliminaire des résultats**

Objectifs du mini-projet

L'objectif principal de ce mini-projet est d'**automatiser un banc de mesure de rayonnement lumineux** à l'aide d'un ordinateur et d'une interface en Python. Le matériel est piloté par une carte à microcontroleur à laquelle il faudra envoyer des commandes selon un protocole série.

Un diagramme de rayonnement lumineux est la **représentation graphique** de la **distribution angulaire** d'une grandeur caractérisant le rayonnement d'une source lumineuse.



Exemple de diagramme de rayonnement d'une LED OSRAM SFH 41747 (documentation technique)

Vous aurez à votre disposition une **maquette** permettant la mise en mouvement d'un système de photodétection autour d'une source de puissance à LED. Cette maquette est pilotée par une carte Nucléo (contenant un microcontroleur).

Un exemple de base (en version archivée et pré-compilée) est disponible au téléchargement ici : https://lense.institutoptique.fr/ressources/mbed/hello-L476RG_pre.zip

Déroulement du bloc

La liste des étapes à suivre pour la réalisation du programme embarqué de la plateforme de rayonnement lumineux est donnée à titre indicatif. L'ordre et le choix des différentes étapes sont laissés à l'appréciation des différents binômes.

Afin de faciliter la réutilisation des codes, il pourra être intéressant de définir des fonctions pour le pilotage des différents éléments.

Séance 1 / MBED et Nucléo-STM32 (sans maquette ! !)

Le sujet de cette séance est fourni dans un document annexe, disponible aussi sur le site du LEnsE - https ://lense.institu dans la rubrique Année / Première Année / Interfaçage Numérique S6 / Bloc 1 Systèmes embarqués / Intro MBED et STM32.

Séance 2 / Prise en main de la maquette et automatisation de la mesure

Etape 6 - 60 min Piloter l'intensité des LEDs de la maquette

Etape 7 - 60 min Piloter le servomoteur de la maquette

Etape 8 - 90 min Définir et tester une première structure de code permettant de piloter le servomoteur en faisant une acquisition de la luminosité à pas régulier

Etape 9 - 60 min Récupérer les données à l'aide d'une liaison Série

Séance 3 / Mise en place d'un protocole de communication

Etape 10 - 90 min Mettre en place un protocole de communication basé sur une liste de commandes et intégrer à la structure du code embarqué

Etape 11 - 90 min Utiliser la bibliothèque pySerial en Python pour envoyer les commandes à la carte Nucléo

Etape 12 - 90 min Tester la communication entre la carte Nucléo et le script Python pour afficher les données

Séance 4 / Application complète

La dernière séance sera consacrée à la finalisation des différents programmes : embarqué sur la carte Nucléo pour la mesure automatique et sur l'ordinateur pour le pilotage de la maquette et l'affichage des données.

Selon le temps restant, il sera possible d'intégrer les fonctions écrites en Python dans une interface graphique, dont la structure de base est fournie.

DISC / Accumulation de preuves

Nous vous rappelons également que l'un des livrables final de ce module est un **Document Individuel de Suivi de Compétences** qui devra revendiquer votre progression dans l'acquisition des compétences suivantes :

— C4 – Valider une solution technologique
— C7 – Travailler en équipe

Travail à réaliser

Vous devrez donc tout au long des séances de TP :

— Documenter les tests réalisés pour valider les fonctionnalités mise en oeuvre
— Stocker les preuves pour revendiquer la montée en compétences (résultats, photos, vidéos...)

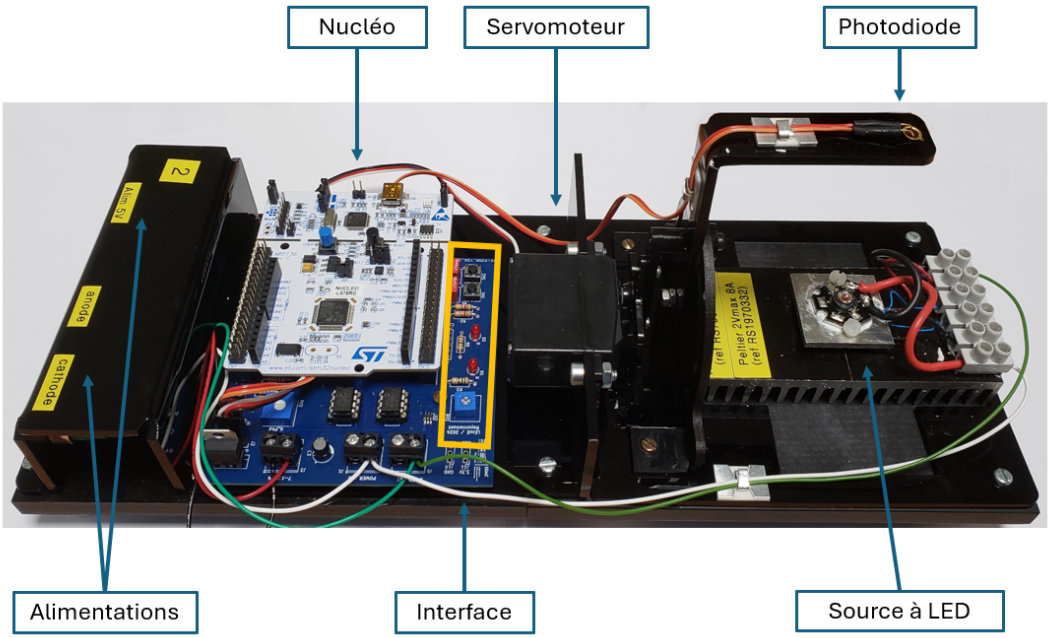
Séance 2 / Prise en main de la maquette et automatisation de la mesure

Objectifs de la séance

Cette seconde séance est consacrée à la **prise en main de la maquette** et à l'automatisation de la mesure de l'intensité lumineuse d'une source à LED (par exemple).

Description de la maquette

La maquette est constituée :
— d'une source de lumière, une LED de puissance
— d'une photodiode BPX65, associée à un circuit de photodétection simple
— d'un servomoteur, permettant de positionner le bras supportant la photodiode au dessus de la source lumineuse selon un angle donné
— d'une carte de contrôle
— d'une carte Nucléo L476RG



Alimentation électrique

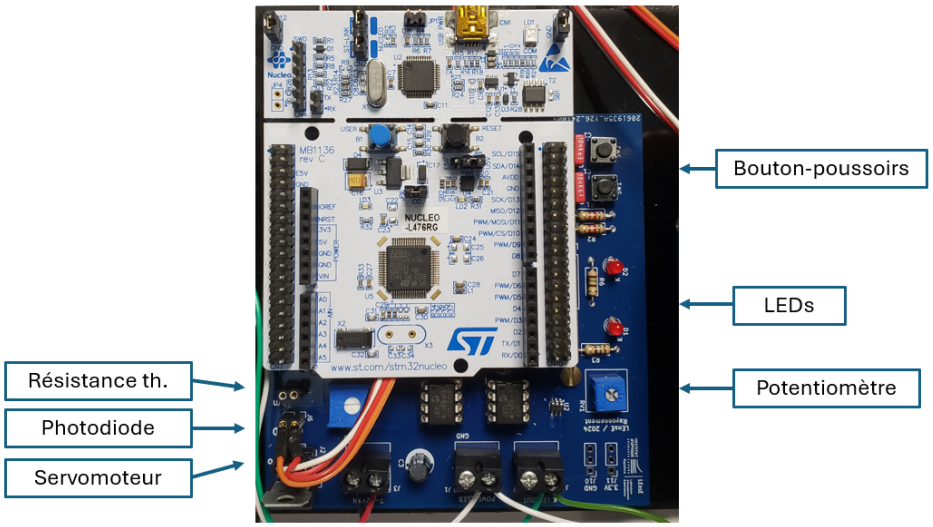
Les bornes d'alimentation notées **Alim** 7 V permet d'alimenter le servomoteur.
Les bornes notées **anode** et **cathode** permettent d'alimenter la LED de puissance.

La tension maximale admissible par le servomoteur est de $7\,\mathrm{V}$!
Le courant maximal admissible par la LED de puissance est de $200\,\mathrm{mA}$!

Brochage / Maquette Rayonnement LED

La **photodiode BPX65** est polarisée. L'**anode** est représentée par un ergot. Sur cette maquette, l'**anode** est reliée par un fil **ORANGE**, la **cathode** par un file **ROUGE**.

Le servomoteur est standard. Sa connectique est constituée de 3 broches : BLANC = signal de pilotage, ROUGE = Alimentation de 5 à 6V, NOIR = masse (GND).



Maquette	**Broche Nucléo**	Type	Description
LED1	PC_7	Sortie / PWM	Led active à l'**état haut**
LED2	PB_13	Sortie / PWM	Led active à l'**état bas**
SW1	PC_6	Entrée	Bouton-poussoir, par défaut état bas
SW2	PC_8	Entrée	Bouton-poussoir, par défaut état bas
USERBUTTON	PC_13	Entrée	Bouton-poussoir, par défaut état haut
SERVO	PB_7	Sortie / PWM	Servomoteur - T = 20 ms
POTENTIOMÈTRE	PC_4	Entrée analogique	Potentiomètre RV1
PHOTODIODE	PC_3	Entrée analogique	Photodiode (montage simple avec R variable)
RÉSISTANCE THERMIQUE	PC_1	Entrée analogique	Résistance Thermique CT10k

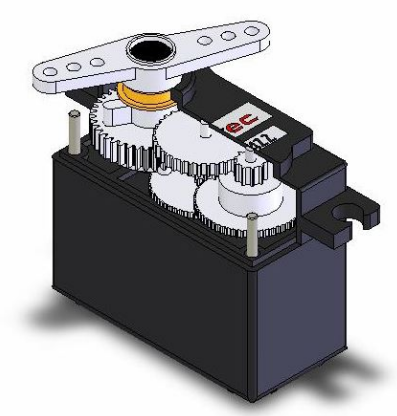
Etape 7 / Piloter l'intensité des LEDs de la maquette

Un exemple de base (en version archivée et pré-compilée) est disponible au téléchargement ici :
https://lense.institutoptique.fr/ressources/mbed/hello-L476RG_pre.zip

➜ **M** A partir de cet exemple, réaliser un programme qui permet de **modifier la luminosité des diodes LED1 et LED2** de la maquette à l'aide des deux bouton-poussoirs SW1 et SW2 (un pour augmenter, l'autre pour diminuer la luminosité). **Votre programme devra utiliser uniquement le principe d'interruption.**

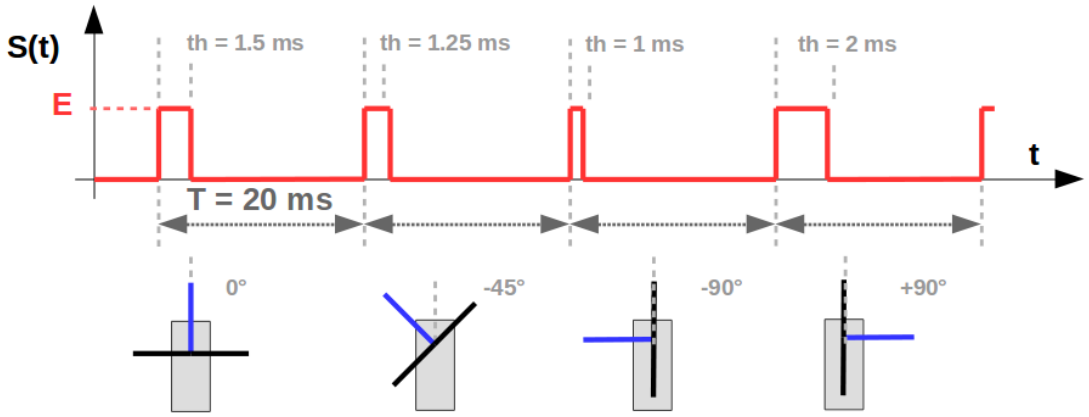
Etape 8 / Piloter le servomoteur de la maquette

Un servomoteur est un actionneur qui réalise une rotation d'un angle calibré en fonction d'une commande externe.



Servomoteur - coupe interne (Crédit : redohm.fr)

Le pilotage d'un servomoteur se fait à l'aide d'un **signal modulé en largeur d'impulsions**. Le signal de commande est un **signal rectangulaire** (numérique) de période fixée à 20 ms. C'est ensuite la durée du temps haut qui permet de modifier l'angle de sortie du servomoteur :



Travail à réaliser

On se propose de tester le code *pwm_servo.cpp*. Ce fichier est disponible sur le site du LEnsE dans la rubrique *Année / Première Année / Interfaçage Numérique S6 / Bloc 1 Systèmes embarqués / Banc de mesure de rayonnement lumineux / Exemples MBED pour STM32*.

➜ **M** Télécharger le projet de base et décompresser l'archive dans un répertoire de votre ordinateur.
https://lense.institutoptique.fr/ressources/mbed/hello-L476RG_pre.zip

➜ **M** Tester le code fourni en remplaçant le contenu du fichier *main.cpp* par le code à tester.

➜ **M** Créer une fonction permettant de positionner le servomoteur à un angle particulier. Tester votre fonction.

Etape 9 / Définir et tester une première structure de code

On cherche à présent à **automatiser l'acquisition des données** de luminosité en fonction de l'angle d'observation.

➜ **M** Réaliser une fonction qui, à partir d'un **angle de départ**, d'un **angle d'arrivée** et d'un **pas angulaire**, permet, pour tous les angles concernés, de :

— déplacer le servomoteur au prochain angle ;

— faire l'acquisition de la luminosité dans un tableau de données.

Le servomoteur ne réagit pas instantanément. Il est donc nécessaire d'ajouter un temps de pause entre le déplacement et l'acquisition.

➜ **M** Tester votre fonction.

Etape 10 / Récupérer les données à l'aide d'une liaison Série

Transmission des données par liaison série

Il est possible de transmettre des données par l'intermédiaire d'une **liaison dite Série** entre le microcontroleur et l'ordinateur.

C'est un protocole asynchrone de transfert de données point à point. Les échanges se font en **Full-Duplex** grâce à deux signaux distincts **RX** (réception) et **TX** (transmission).

Cette fonctionnalité a déjà été utilisée dans l'étape 5 pour afficher sur l'ordinateur (logiciel TeraTerm) les données converties à l'aide du convertisseur analogique numérique.

➜ **M** A partir du programme de l'étape 5 et des autres programmes que vous avez mis au point, réaliser une fonction qui permet de transmettre par la liaison série les valeurs sauvegardées dans le tableau de données précédemment acquis.

➜ **M** Tester votre fonction.

Pilotage de la LED de puissance

➜ **M** Alimenter la LED de puissance avec un courant maximal de 200mA.

Attention ! Paramètrer l'alimentation de la LED pour ne pas dépasser 200mA !
Attention également à vos yeux...
Et brancher la LED dans le bon sens... Cathode = point de plus faible potentiel.

Premier diagramme de rayonnement

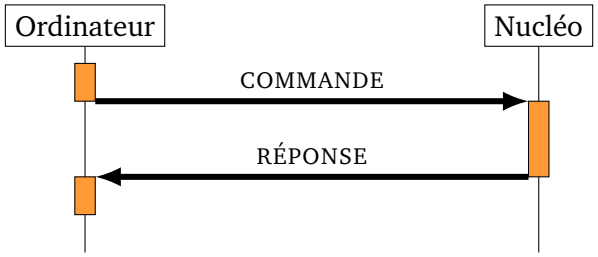
➜ **M** Faire l'acquisition des valeurs d'intensité lumineuse pour des angles allant de 0 à $180°$ par pas de $10°$ et transmettre les données par liaison Série à l'ordinateur.

Séance 3 / Mise en place d'un protocole de communication

Objectifs de la séance

Cette séance a pour but de mettre en place un **protocole de communication** entre la carte embarquée et l'ordinateur, d'abord à l'aide d'un moniteur série puis à l'aide d'un script en Python.

L'ordinateur, maitre de la communication, devra envoyer des commandes à la carte embarquée, qui devra les exécuter puis acquitter du bon déroulement de l'opération.



Un exemple de code pour la transmission des commandes A et D, à analyser et à tester, est fourni.

Protocole de communication

Un **protocole de communication** est un ensemble de règles, de conventions et de formats définis pour permettre l'échange d'informations entre différents systèmes. Il détermine comment les données sont structurées, envoyées, reçues et interprétées par les parties communicantes.

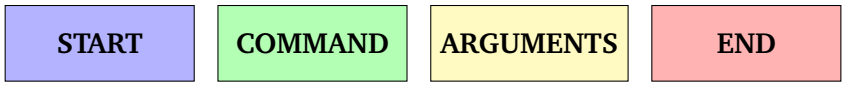
Il est important lorsqu'on souhaite **construire un protocole de communication** de :
- **choisir un protocole de bas niveau** pour la transmission des commandes et des réponses
- définir la **structure des commandes**
- planifier la **liste des commandes**
- coder la récupération des données sur la carte Nucléo (et tester avec un moniteur Série)
- coder l'envoi des commandes sur le PC (en Python par exemple)

Choix du protocole de bas niveau

Nous utiliserons dans le cas de notre application la **liaison Série** (ou RS232) qui nous a déjà servi à afficher des données depuis la carte Nucléo.

Structure des commandes

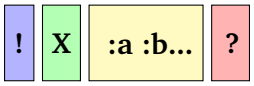
Chaque commande doit avoir un **format bien défini** pour que le microcontroleur puisse la comprendre et la traiter.



La liaison Série étant asynchrone, il est indispensable de prévoir dans le protocole des **caractères de début et de fin** de chaines de commande et de réponse.

Commande

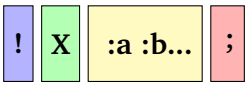
Par exemple, on pourra se baser sur la structure suivante pour le cas d'une commande (entre le PC et le microcontroleur) :



Les arguments sont facultatifs, selon les commandes.

Réponse

Par exemple, on pourra se baser sur la structure suivante pour le cas d'une réponse (entre le PC et le microcontroleur) :



Les arguments sont facultatifs, selon les commandes.

Liste des commandes

Commande PC	Description	**Réponse Nucléo**
!T ?	Test de la transmission	!T ;
!A :VALUE ?	Transmission de l'angle de départ souhaité pour le diagramme valeur entière en degré *Si angle non compris entre -90 et 90*	!A :VALUE ; VALUE= -100
!B :VALUE ?	Transmission de l'angle final souhaité pour le diagramme valeur entière en degré *Si angle non compris entre -90 et 90*	!B :VALUE ; VALUE= -100
!C :VALUE ?	Transmission du pas angulaire souhaité pour le diagramme valeur entière en degré *Si angle non compris entre -90 et 90*	!C :VALUE ; VALUE= -100
!S ?	Lancement de l'acquisition nb est le nombre d'échantillons qui seront acquis par la carte *Si les angles fournis sont non compatibles*	!S :NB ; NB= 0
!E ?	Test de fin de l'acquisition (Y)es or (N)o	!E :Y/N ;
!D :INDEX ?	Demande de récupération d'une donnée index correspond au numéro de l'échantillon souhaité value correspond à la valeur de l'échantillon souhaité	!D :INDEX :VALUE ;

Etape 11 / Mettre en place un protocole de communication basé sur une liste de commandes

Envoi et réception de messages simples

Avant de se lancer dans la mise en place du protocole complet, nous vous proposons de tester l'exemple donné dans le fichier *test_comm_led.cpp*. Cet exemple se base sur un protocole simplifié, ne nécessitant l'envoi que d'**un seul caractère par commande**.

➜ **M** Tester le code fourni en envoyant des lettres par l'intermédiaire d'une console Série (logiciel **TeraTerm** par exemple).

➜ **Q** A quel moment est appelée la fonction *ISR_my_pc_reception()* ? A quoi sert-elle ?

On propose de remplacer la console Série par un **script Python** contenu dans le fichier *test_comm_led.py*.

➜ **M** Exécuter ce script python.

Structure de base

On se propose de tester le code *test_version_a_d.cpp*. Ce fichier est disponible sur le site du LEnsE dans la rubrique *Année / Première Année / Interfaçage Numérique S6 / Banc de mesure de rayonnement lumineux / Exemples de Codes*.

➜ **M** Compiler ce code et le téléverser sur la carte Nucléo.

➜ **Q** A quoi sert la fonction *is_angle_ok()* ?

➜ **M** Ouvrir le logiciel TeraTerm et envoyer la commande suivante : !A :10 ?.

➜ **Q** Que renvoie la carte Nucléo ? Expliquer les fonctions *ISR_my_pc_reception()* (vis-à-vis du choix de la structure d'une commande) et la boucle infinie. On s'intéressera notamment aux changements de valeur des variables *string_complete* et *input_cnt*.

➜ **Q** Que se passe-t-il lorsqu'on transmet une mauvaise valeur (soit avec la commande A, soit avec la commande D) ?

➜ **Q** Faire un algorigramme du code.

Ajout des autres commandes

On souhaite à présent compléter ce code pour pouvoir traiter l'ensemble des commandes mentionnées dans la section précédente (T, A, B, C, S, E, D).

➜ **M** Compléter le code précédent et le tester.

➜ **Q** A quel moment doit-on utiliser la **fonction d'acquisition** réalisée à l'étape 8 ? D'autres commandes sont-elles acceptables lors de l'acquisition ? Comment bloquer l'exécution des autres commandes ?

➜ **M** Mettre en place ce blocage.

➜ **Q** A quel moment doit-on utiliser la **fonction de transmission** des données réalisée à l'étape 9 ?

Test complet

➜ **Q** Quelle est la suite de commandes à transmettre à la carte Nucléo pour permettre l'acquisition de données pour des angles allant de $-40°$ à $+60°$ par pas de $10°$ et la récupération des données sur le moniteur Série ?

➜ **Q** Tracer le diagramme de séquence associé à ces échanges de commandes et de réponses entre le PC et la carte Nucléo.

➜ **M** Tester l'acquisition de cette série de données.

Etape 12 / Utiliser la bibliothèque pySerial en Python pour envoyer les commandes à la carte Nucléo

> **ATTENTION** / Lors de l'utilisation d'un script Python nécessitant l'accès à la liaison Série, le logiciel **TeraTerm** doit être **fermé**.

On se propose à présent d'utiliser un script en langage Python pour pouvoir transmettre les commandes et recevoir les données sur l'ordinateur. Nous utiliserons la bibliothèques **pySerial** (à installer par la commande *pip install pyserial* dans une console ou un prompt d'Anaconda).

Un code de base, nommé *test_version_a_d.py*, est disponible sur le site du LEnsE dans la rubrique *Année / Première Année / Interfaçage Numérique S6 / Banc de mesure de rayonnement lumineux / Exemples de Codes*.

➜ **M** Exécuter ce code et tester les deux commandes pré-configurées ('a' et 'd').

➜ **M** A partir de cet exemple, réaliser un script qui permette de lancer l'acquisition de données pour des angles allant de $-40°$ à $+60°$ par pas de $10°$ et la récupération des données acquises dans une liste en Python.

➜ **M** Afficher les données obtenues sur un graphique à l'aide de Matplotlib. Générer l'axe des angles.

Etape 13 / Tester la communication entre la carte Nucléo et le script Python pour afficher les données

On souhaite enfin créer une application en console complète permettant à l'utilisateur de saisir les angles de départ et de fin, le pas angulaire et de lancer l'acquisition. Votre application devra ensuite afficher les données acquises.

Vous pourrez faire plusieurs essais avec des courants dans la LED de puissance de $50\,\mathrm{mA}$, $100\,\mathrm{mA}$ et $200\,\mathrm{mA}$ par exemple.

Interfaçage Numérique

Travaux Pratiques

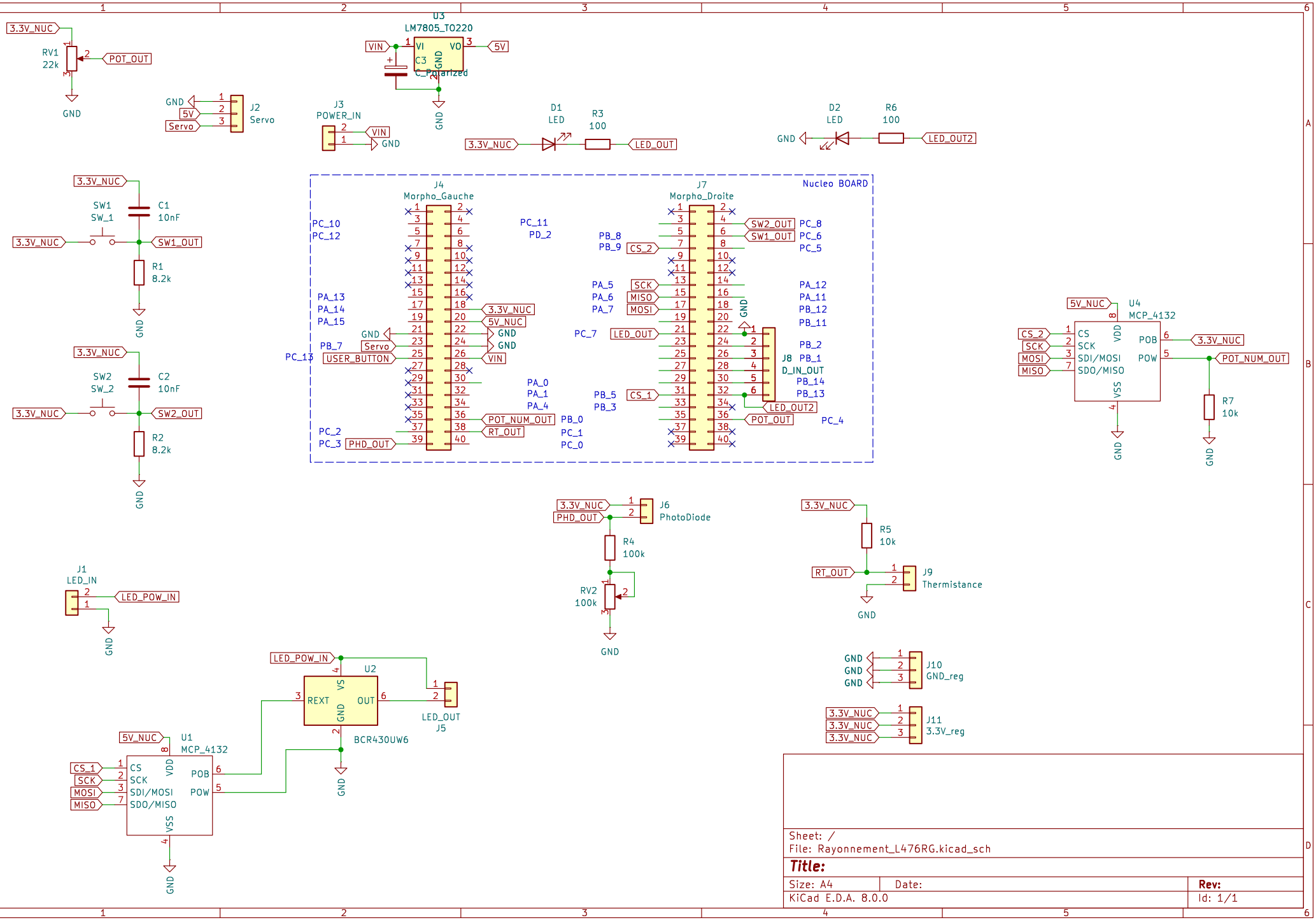
Semestre 6

Ressources

Bloc Rayonnement

Liste des ressources

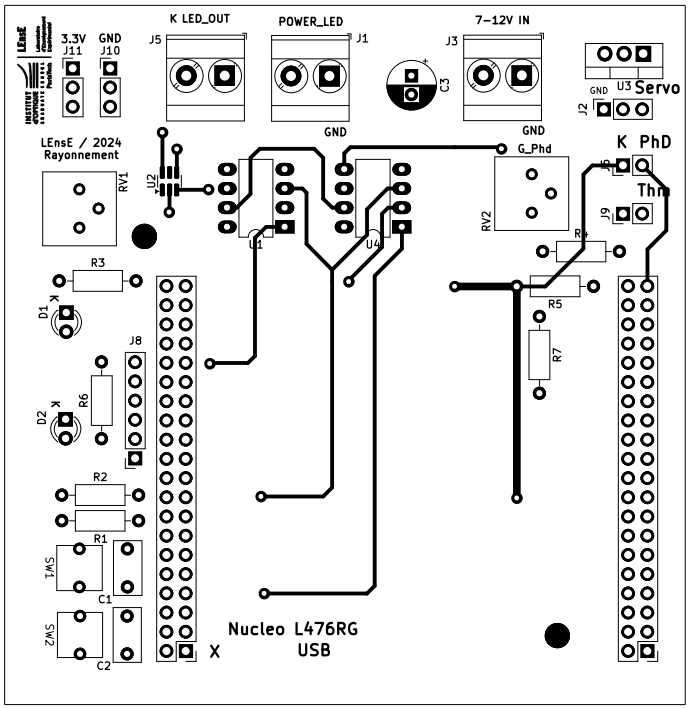
— Schéma de la carte de la maquette de rayonnement
— PCB de la carte de la maquette de rayonnement



Sheet: /
File: Rayonnement_L476RG.kicad_sch

Title:

Size: A4	Date:	**Rev:**
KiCad E.D.A. 8.0.0		Id: 1/1


K LED_OUT
POWER_LED
7-12V IN
3.3V
GND
J11
J10
J5
J1
J3
C3
U3
Servo
GND
J2
GND
GND
LEnsE / 2024
Rayonnement
RV1
U2
U1
U4
G_Phd
RV2
K PhD
Thr
J9
R4
R3
R5
R7
D1
J8
D2
R6
R2
R1
SW1
C1
SW2
C2
Nucleo L476RG
X
USB